Guia Pràctica: Control de Versions amb Git i Fluxos de Treball en GitHub

Apunts de classe i manual de referència tècnica sobre configuració d'entorns de desenvolupament, gestió de repositoris locals i remots, desplegament de llocs web estàtics i automatització de proves amb GitHub Actions.

1. Configuració Inicial de l'Entorn de Treball

Aquest bloc recull els passos necessaris per establir la identitat global de desenvolupament i l'autenticació mitjançant tokens d'accés personal a la consola de GitHub.

1.1 Identitat Global de Git

Abans de realitzar qualsevol confirmació (*commit*), és imprescindible establir el nom i correu electrònic oficials que s'associaran a l'historial de desenvolupament:git config --global user.name "Nom Cognom"

git config --global user.email "correu@exemple.com"

1.2 Autenticació amb Personal Access Token (PAT)

Per configurar l'autenticació remota segura sense dependre de la contrasenya tradicional del compte:

1. Accedir a GitHub > **Settings** > **Developer Settings** > **Personal access tokens (classic)**.
2. Crear un token seleccionant tots els abasts d'accés requerits (`repo`, `workflow`, `write:packages`).
3. Desar el token generat en un fitxer anomenat `auth.txt` i executar les següents comandes d'autenticació amb la CLI de GitHub:

gh auth login-with-token < auth.txt

gh auth setup-git

2. Gestió Basal de Repositoris i Registre d'Historial

Aquest apartat descriu com estructurar un projecte des de zero, vincular-lo amb un repositori remot i generar un registre exportable de l'historial de canvis.

2.1 Creació i Vinculació Local-Remota

1. Crear el repositori públic `bloc-1` a GitHub (establint la branca per defecte com a `main`).
2. Inicialitzar l'entorn de treball local i assignar el directori remot corresponent:

```
mkdir -p ~/projectes/bloc-1

cd ~/projectes/bloc-1

git init -b main

git remote add origin https://github.com/<usuari>/bloc-1.git
```

2.2 Primers Commits i Exportació del Log

1. Crear el fitxer de documentació inicial `respostes.md` i dur a terme el primer enviament a la branca remota:

```
echo "# Respostes Nom Cognom" > respostes.md

git add respostes.md

git commit -m "Primer commit"

git push -u origin main
```

2. Crear la documentació principal `Readme.md` i registrar el segon commit local:

```
echo "Nom Cognom" > Readme.md

git add Readme.md

git commit -m "Segon commit"
```

3. Generar el fitxer d'historial resumit `log.txt`, consolidar el tercer commit i sincronitzar amb el servidor remot:

```
git log --oneline > log.txt

git add log.txt

git commit -m "Tercer commit"
```

git push

3. Gestió Selectiva de Canvis (Staging) i Llocs Web Simples

Tècniques per organitzar modificacions independents en la zona de preparació (*staging area*) abans de la seva confirmació final.

3.1 Estructura del Projecte Web

1. Crear el repositori `bloc-2` a GitHub i preparar el directori de treball local:

mkdir -p ~/projectes/bloc-2

cd ~/projectes/bloc-2

git init -b main

git remote add origin https://github.com/<usuari>/bloc-2.git

2. Crear el fitxer d'entrada principal `index.html`:

<!DOCTYPE html>

<html lang="ca">

<head>

<meta charset="utf-8">

<title>Portfoli de Nom Cognom</title>

<link rel="stylesheet" href="estil.css">

</head>

<body>

<h1>Portfoli de Nom Cognom</h1>

Pàgina publicada amb GitHub Pages.

</body>

</html>

3.2 Commits Granulars

1. Indexar **únicament** la pàgina principal i registrar la primera modificació:

git add index.html

git commit -m "Afegeix pàgina principal"

2. Afegir el fitxer `Readme.md` amb el contingut `# Capçalera`, indexar-lo de manera aïllada i confirmar el canvi:

echo "# Capçalera" > Readme.md

git add Readme.md

git commit -m "Afegeix readme"

3. Enviar l'historial complet de commits al repositori remot:

git push -u origin main

4. Exclusió d'Arxius i Regles de `.gitignore`

Mètodes per prevenir la publicació no desitjada de fitxers temporals, directoris de registre (*logs*) o informació sensible de l'entorn.

4.1 Configuració del Fons d'Exclusions

1. Accedir al directori de treball del projecte `bloc-3`.
2. Definir l'exclusió explícita de la carpeta de registres `logs/`:

echo "logs/" > .gitignore

3. Inicialitzar el repositori `bloc-3` a GitHub i configurar el vincle remot:

git init -b main

git remote add origin https://github.com/<usuari>/bloc-3.git

4.2 Ordre d'Inclusió dels Canvis

1. Confirmar i publicar en primer lloc el fitxer de regles d'exclusió:

git add .gitignore

git commit -m "Afegeix .gitignore"

git push -u origin main

2. Afegir el fitxer de codi font `script.py` en un segon commit posterior:

git add script.py

git commit -m "Afegeix script"

git push

5. Publicació d'Alineacions Estàtiques a GitHub Pages

Procediment pas a pas per a l'allotjament automatitzat de portals i aplicacions web estàtiques mitjançant el servei integrat GitHub Pages.

5.1 Configuració del Repositori d'Usuari

1. Crear el repositori públic especial amb la nomenclatura `<usuari>.github.io`.
2. Extreure el contingut base al directori local i publicar la primera versió del lloc:

git init -b main

git remote add origin https://github.com/<usuari>/<usuari>.github.io.git

git add .

git commit -m "Subir contenido inicial bloc-4"

git push -u origin main

5.2 Habilitació del Servei i Actualització

1. A la interfície web de GitHub, navegar a **Settings** > **Pages** > **Branch** i seleccionar la branca `main` a l'arrel `/ (root)`.
2. Un cop confirmada la publicació del lloc web, actualitzar el fitxer `README.md` afegint el nom complet al final del document:

echo "Nom Cognom" >> README.md

git add README.md

git commit -m "Afegeix el meu nom"

git push

6. Integració Contínua (CI) amb GitHub Actions

Implementació de fluxos de treball automatitzats (*workflows*) per a l'execució de proves unitàries en una matriu multi-sistema operatiu (Windows, Ubuntu Linux i macOS).

6.1 Estructura del Flux de Treball

1. Preparar el projecte `bloc-5` i vincular-lo amb el seu corresponent repositori remot a GitHub.
2. Crear la llista de directoris reservats per a les definicions d'automatització:

mkdir -p .github/workflows

3. Definir el fitxer de configuració del workflow a `.github/workflows/unittest.yml`:

name: Python Unit Tests

on:

  push:

    branches:

      - main

jobs:

  test-win:

    runs-on: windows-latest

    steps:

      - name: Checkout code

        uses: actions/checkout@v6

      - name: Set up Python 3.12

        uses: actions/setup-python@v6

```
      with:
        python-version: '3.12'
    - name: Run unit tests
      run: python -m unittest -v
test-linux:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v6
    - name: Set up Python 3.13
      uses: actions/setup-python@v6
      with:
        python-version: '3.13'
    - name: Run unit tests
      run: python -m unittest -v
test-mac:
  runs-on: macos-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v6
    - name: Set up Python 3.13
      uses: actions/setup-python@v6
```

```
      with:
        python-version: '3.13'
    - name: Run unit tests
      run: python -m unittest -v
```

6.2 Desplegament del Workflow

Publicar el directori complet al repositori remot per activar l'execució automàtica de la bateria de tests